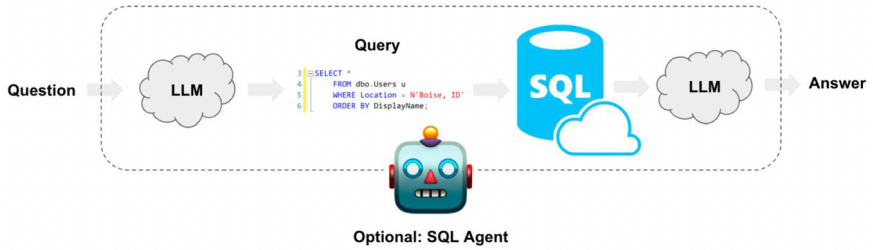
Text-to-SQL with LLMs



Learning objectives

- Connect LangChain's SQLDatabase to a local SQLite file (Chinook sample) and use get_table_info() to feed schema context to prompts.
- Explain what **SQLDatabaseChain** does (**built-in prompts, question → LLM SQL → db.run → optional LLM answer)** and use OllamaLLM with **return_intermediate_steps**, **top_k,** and optional **use_query_checker**.
- Contrast the chain with **manual ChatOllama**: why custom prompts, regex + db.run, then optional error-driven retries (Part 3 vs. Part 4 in the notebook).
- Optionally **add few-shot in-context examples** (n = 2, 3, or 4 question/SQL pairs) in the same prompt to **steer** small models.
- Implement a **small plain-Python generate → run → (optional) repair loop** without extra orchestration libraries.
- Recognize **limitations** of small local models and **unsafe SQL**: read-only practice DB only; do not point at production data without safeguards.

Provided files (this exercise folder)

- data/Chinook.db — created by the notebook from the public Chinook SQLite script (music store: artists, albums, tracks, …).
- Text_to_SQL_with_LLMs.ipynb — main lab: setup, SQLDatabase, SQLDatabaseChain, manual chat + few-shot (optional), retry loop.
- requirements-text-to-sql-exercise.txt — suggested pip dependencies.

Part 0 — Environment and data

Install packages (see notebook's first code cell), start Ollama, and pull a small chat/completion model (e.g. llama3.2:1b) plus optional larger model if SQL quality is weak.

```
# Example (in notebook)
%pip install -qU "sqlalchemy>=2" langchain-core langchain-community
langchain-ollama langchain-experimental python-dotenv
```

The notebook downloads the Chinook schema script and builds data/Chinook.db if the file is missing. Manual alternative:

```
curl -s "https://raw.githubusercontent.com/lerocha/chinook-
database/master/ChinookDatabase/DataSources/Chinook_Sqlite.sql" |
sqlite3 Chinook.db
```

Part 1 — SQLDatabase: tables and schema text

List tables, print a prefix of get_table_info() (column names and types for prompts), and run a few sample SELECTs including a simple GROUP BY to verify JOINs and aggregates work.

```
from langchain_community.utilities import SQLDatabase
db = SQLDatabase.from_uri("sqlite:///data/Chinook.db")
print(len(db.get_usable_table_names()))
print(db.get_table_info()[:2000])
print(db.run("SELECT COUNT(*) FROM Track;"))
```

Part 2 — What SQLDatabaseChain is and what it does

SQLDatabaseChain (langchain_experimental) is a pre-built chain: it connects a text LLM to a SQLDatabase, uses a built-in prompt that injects schema from the database, and typically runs: natural language question → model generates SQL → the chain executes the SQL on the connection → the result string is optionally turned into a short natural-language answer by a second call to the model. You invoke with invoke({"query": "..."}) and the default output is that final text answer. OllamaLLM is a completion model (not ChatOllama) for the usual from_llm setup.

Key flags:

- verbose=True (log intermediate text),
- return_intermediate_steps=True (get generated SQL in Python),
- use_query_checker=True (extra pass to try to fix SQL before run),
- top_k = max rows of query result used when wording the final answer.

The chain is a good baseline; it hides prompt details, which motivates Parts 3–4 in the notebook.

```
from langchain_ollama import OllamaLLM
from langchain_experimental.sql import SQLDatabaseChain

llm = OllamaLLM(model="llama3.2:1b", temperature=0)
db_chain = SQLDatabaseChain.from_llm(
    llm, db, verbose=True, return_intermediate_steps=True,
use_query_checker=False, top_k=8
```

```
)
out = db_chain.invoke({"query": "How many artists are there?"})
print(out["result"])
# print(out.get("intermediate_steps"))
```

Weaker models may output invalid SQL. Try a larger model, shorter schema context, a clearer user question, or the manual chat + db.run path in the notebook.

Part 3 — Why manual ChatOllama and db.run

Part 3 is there so you see the "**you own the full prompt and execution**" pattern:

system + human messages → get_table_info() for schema→ regex to extract a single SELECT, → db.run.

That is closer to product scenarios with guardrails, logging, or tool use than the one-shot chain.

The notebook also includes an optional few-shot block: n = 2, 3, or 4 (question, SQL) example pairs in the same prompt to improve local models (in-context learning, more tokens).

Part 4 — Why a separate retry loop

Part 3 is usually a single model turn before execution. When SQLite returns an error, a practical pattern is to call the model again with the error text (and optionally cap retries).

The chain can use use_query_checker for a related idea, but Part 4 makes the control flow explicit in plain Python (ask_chinook_with_retry).

```
# See notebook: ask_chinook_with_retry -> extract_select -> db.run,
loop on exception
```

Follow-up

- Run 2–3 different questions: one aggregate, one JOIN across 2+ tables, one sort/limit ("top 5 …").
- Compare `SQLDatabaseChain` vs. manual `ChatOllama` + `db.run` when the model is weak: which path fails first?

- (Optional) Turn few-shot `N` from `2` to `4` on the same new question: does SQL quality or stability improve? Report token/latency tradeoff in one line.

- Try a larger model (if you can), shorten `get_table_info()`, or name specific tables in the user question.

References

- Chinook database — https://github.com/lerocha/chinook-database (SQLite script under DataSources).
- SQLDatabase (LangChain) — https://python.langchain.com/docs/integrations/tools/sqlite/